ERP-SQL

Using MYSQL Workbench

Multiple Tables

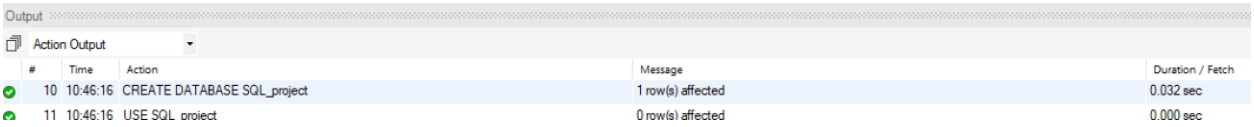
Instead of one table, companies use multiple related tables:

- Multiple tables (normalized design)
- JOINs
- Advanced SQL (GROUP BY, HAVING, subqueries, window functions)

CREATE DATABASE:

```
CREATE DATABASE SQL_project;
USE SQL_project;

Output:
```



1. Create Additional Tables (Real-World Structure)

Instead of one table, companies use multiple related tables:

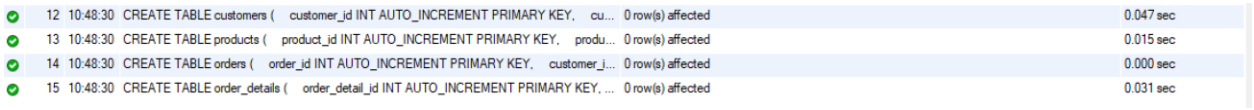
```
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50)
);

CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(50),
    category VARCHAR(50),
    price DECIMAL(10,2)
);

CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

```
CREATE TABLE order_details (
    order_detail_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

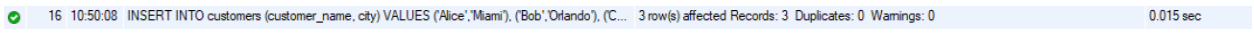
Output:



2. Sample Data (Short Version)

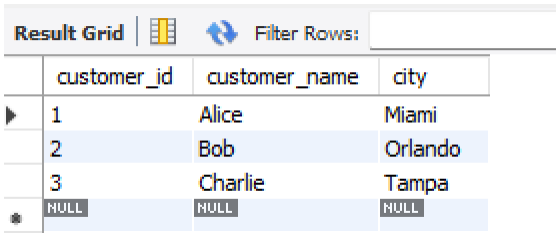
```
-- Customers
INSERT INTO customers (customer_name, city) VALUES
('Alice','Miami'),
('Bob','Orlando'),
('Charlie','Tampa');
```

Output:



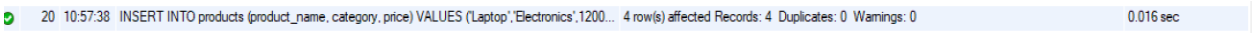

```
SELECT * FROM customers
```

Output:




```
-- Products
INSERT INTO products (product_name, category, price) VALUES
('Laptop','Electronics',1200),
('Phone','Electronics',800),
('Shoes','Fashion',60),
('Watch','Accessories',150);
```

Output:




```
SELECT * FROM products
```

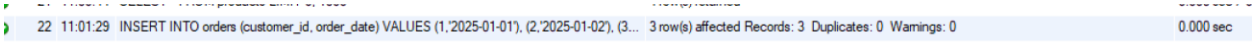
Output:

Result Grid | Filter Rows: | Edit:

product_id	product_name	category	price
1	Laptop	Electronics	1200.00
2	Phone	Electronics	800.00
3	Shoes	Fashion	60.00
4	Watch	Accessories	150.00
NULL	NULL	NULL	NULL

```
-- Orders
INSERT INTO orders (customer_id, order_date) VALUES
(1,'2025-01-01'),
(2,'2025-01-02'),
(3,'2025-01-03');
```

Output:




```
SELECT * FROM orders
```

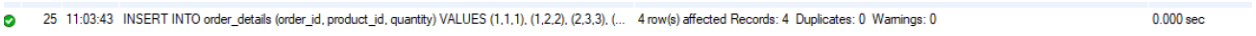
Output:

Result Grid | Filter Rows:

order_id	customer_id	order_date
1	1	2025-01-01
2	2	2025-01-02
3	3	2025-01-03
NULL	NULL	NULL

```
-- Order Details
INSERT INTO order_details (order_id, product_id, quantity) VALUES
(1,1,1),
(1,2,2),
(2,3,3),
(3,4,1);
```

Output:




```
SELECT * FROM order_details
```

Output:

Result Grid | Filter Rows: | Ed

order_detail_id	order_id	product_id	quantity
1	1	1	1
2	1	2	2
3	2	3	3
4	3	4	1
NULL	NULL	NULL	NULL

3.JOIN Examples (Core Skill)

INNER JOIN (Most Important)

```
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    od.quantity,
    p.price
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id;
```

Output:

Result Grid | Filter Rows: | Export:

order_id	customer_name	product_name	quantity	price
1	Alice	Laptop	1	1200.00
1	Alice	Phone	2	800.00
2	Bob	Shoes	3	60.00
3	Charlie	Watch	1	150.00

LEFT JOIN (Show all customers even without orders)

```
INSERT INTO customers (customer_name, city) VALUES
('Emma','Wesley Chapel'),
('Jon','Brooksville'),
('Rex','SPring Hill');
```

Output:

```
28  11:25:46  INSERT INTO customers (customer_name, city) VALUES ('Emma','Wesley Chapel'), ('Jon','Br...  3 row(s) affected Records: 3  Duplicates: 0  Warnings: 0  0.031 sec
```

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Output:

Result Grid | Filter Rows:

	customer_name	order_id
▶	Alice	1
	Bob	2
	Charlie	3
	Emma	NULL
	Jon	NULL
	Rex	NULL

4. Business-Level Queries (Real Job Tasks)

Total Revenue per Customer

```
SELECT
    c.customer_name,
    SUM(p.price * od.quantity) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
GROUP BY c.customer_name
ORDER BY total_spent DESC;
```

Output:

Result Grid | Filter Rows:

	customer_name	total_spent
▶	Alice	2800.00
	Bob	180.00
	Charlie	150.00

Top Selling Products

```
SELECT
    p.product_name,
    SUM(od.quantity) AS total_sold
FROM order_details od
JOIN products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC;
```

Output:

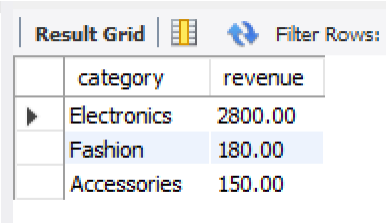
Result Grid | Filter Rows:

	product_name	total_sold
▶	Shoes	3
	Phone	2
	Laptop	1
	Watch	1

Revenue by Category

```
SELECT
    p.category,
    SUM(p.price * od.quantity) AS revenue
FROM products p
JOIN order_details od ON p.product_id = od.product_id
GROUP BY p.category;
```

Output:



Result Grid | Filter Rows:

category	revenue
Electronics	2800.00
Fashion	180.00
Accessories	150.00

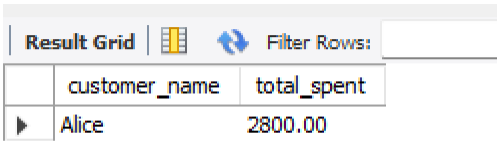
5. HAVING (Advanced Filtering)

👉 Difference:

- WHERE filters rows
- HAVING filters aggregated results

```
SELECT
    c.customer_name,
    SUM(p.price * od.quantity) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
GROUP BY c.customer_name
HAVING total_spent > 1000;
```

Output:



Result Grid | Filter Rows:

customer_name	total_spent
Alice	2800.00

6. Subquery (Very Common in Interviews)

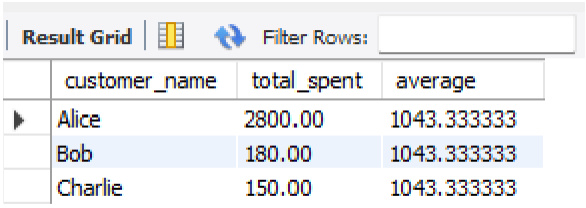
Customer total spending and customers average spending

```
SELECT
      c.customer_name,
      SUM(p.price * od.quantity) AS total_spent,
      (SELECT AVG(total_spent)
          FROM (
```

```
            SELECT SUM(p.price * od.quantity) AS total_spent
            FROM orders o
            JOIN order_details od ON o.order_id = od.order_id
            JOIN products p ON od.product_id = p.product_id
            GROUP BY o.customer_id
        ) avg_table) ave_spent

    FROM orders o
    JOIN customers c ON c.customer_id = o.customer_id
    JOIN order_details od ON o.order_id = od.order_id
    JOIN products p ON od.product_id = p.product_id
    GROUP BY c.customer_name
```

Output:



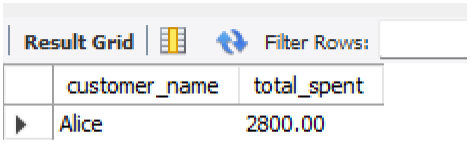
Result Grid | Filter Rows:

customer_name	total_spent	average
Alice	2800.00	1043.333333
Bob	180.00	1043.333333
Charlie	150.00	1043.333333

Customers who spent above average

```
SELECT customer_name, total_spent
FROM (
    SELECT
        c.customer_name,
        SUM(p.price * od.quantity) AS total_spent
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_details od ON o.order_id = od.order_id
    JOIN products p ON od.product_id = p.product_id
    GROUP BY c.customer_name
) AS customer_totals
WHERE total_spent > (
    SELECT AVG(total_spent)
    FROM (
        SELECT SUM(p.price * od.quantity) AS total_spent
        FROM orders o
        JOIN order_details od ON o.order_id = od.order_id
        JOIN products p ON od.product_id = p.product_id
        GROUP BY o.customer_id
    ) avg_table
);
```

Output:



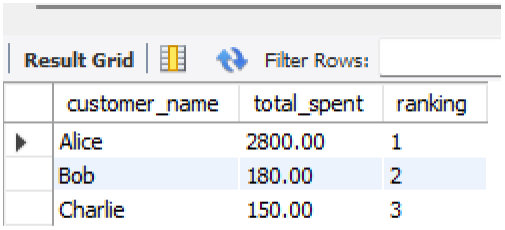
Result Grid | Filter Rows:

customer_name	total_spent
Alice	2800.00

7. Window Function (Advanced / Real Analyst Level)

Rank customers by spending

```
SELECT
    c.customer_name,
    SUM(p.price * od.quantity) AS total_spent,
    RANK() OVER (ORDER BY SUM(p.price * od.quantity) DESC) AS ranking
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
GROUP BY c.customer_name;
```



	customer_name	total_spent	ranking
▶	Alice	2800.00	1
	Bob	180.00	2
	Charlie	150.00	3

Designed a relational MySQL database with normalized tables (customers, products, orders, order_details). Performed complex data analysis using JOINs, aggregations, subqueries, HAVING clauses, and window functions to generate insights such as customer lifetime value(Rank customers by spending), top-selling products, and revenue trends.